



2 📖

10 📄

1200 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

# Hyper Text Transfer Protocol (HTTP)

د. غاندي هسام



**RB** Informatics; 15/10/2025

لغات البرمجة

## البروتوكول:

مجموعة من القواعد و التعليمات التي تحدد كيف تتواصل الأجهزة او البرامج مع بعضها البعض. عندما يتحدث شخصان ، يجب ان يتحدثوا بنفس اللغة ليفهموا بعض، و كذلك في عالم الحواسيب ، الأجهزة تحتاج لغة مشتركة لتنقل البيانات بينها و هذه " اللغة المشتركة " تسمى بروتوكول و هي التي تحدد كيف يُرسل و يُستقبل كل جزء من البيانات خطو بخطوة.

### ✍ مثال بسيط:

إذا أرسلت رسالة من هاتفك الى صديقك عبر واتساب:

- جهازك لا يرسل الكلام كما هو
- يقسم الرسالة الى بيانات Data ، و يضيف معلومات مثل : العنوان ، نوع الرسالة، حجمها ....
- ثم ترسل عبر بروتوكولات معينة

و صديقك يستقبل الرسالة بنفس القواعد ، فيعيد البرنامج ترتيبها و يعرضها بالشكل الصحيح.

### ✍ من اهم البروتوكولات:

| الوظيفة                                              | البروتوكول     |
|------------------------------------------------------|----------------|
| لتصفح الإنترنت و نقل صفحات الويب                     | HTTP/ HTTPS    |
| لنقل الملفات بين الأجهزة                             | FTP            |
| لإرسال و استقبال البريد الالكتروني                   | SMTP/POP3/IMAP |
| لنقل البيانات بين الأجهزة بطريقة مضمونة و سريعة      | TCP/UDP        |
| لتوجيه البيانات عبر الانترنت (يعطي كل جهاز عنوان IP) | IP             |



## HTTP(HyperText Transfer Protocol):

هو بروتوكول لتبادل الملفات و البيانات عبر الويب (مثل صفحات HTML ، الصور ، الفيديوهات...).

### ✚ فكرته الأساسية:

- يعتمد على نظام العميل و الخادم (Client – Server)
- العميل: المتصفح (مثل Chrome أو Firefox).
- الخادم: السيرفر الذي يحتوي الملفات و الموقع.
- العميل يرسل طلب (Request) إلى السيرفر، و السيرفر يرد عليه برد (Response).

### ✚ مثال:

عندما تكتب رابط موقع URL و تضغط Enter ، المتصفح يرسل طلب HTTP للسيرفر ، و السيرفر يرسل الصفحة كاستجابة.

ملاحظة: العميل (Client) و السيرفر يتواصلان من خلال تبادل رسائل مستقلة ، و ليس تدفق مستمر من البيانات ، يعني كل طلب و رد منفصل عن الآخر.

### العلاقة بين HTTP و طبقة النقل (TCP / TLS):

بروتوكول HTTP يعمل في طبقة التطبيق (application layer) و يُرسل عبر TCP و هو بروتوكول نقل موثوق او عبر TLS-encrypted TCP (أي نسخة مشفرة منه HTTPS).

ملاحظة: ما المقصود بـ application layer ؟

هي الطبقة التي تتعامل مع البرامج و التطبيقات مباشرة مثل : المتصفحات (chrome,firefox) و تطبيقات البريد الالكتروني... فهي اعلى طبقة في نموذج الاتصال.

### ✚ كيف يرسل HTTP البيانات ؟

HTTP لا يرسل البيانات مباشرة بل يستخدم TCP كوسيلة نقل.

#### TCP

#### Transmission Control Protocol

يتأكد أن:

- كل البيانات تصل الى الوجهة .
- لا يوجد فقد في الرسائل.
- يتم ترتيبها بنفس الترتيب الذي أرسلت به.

#### TLS

#### Transport Layer Security

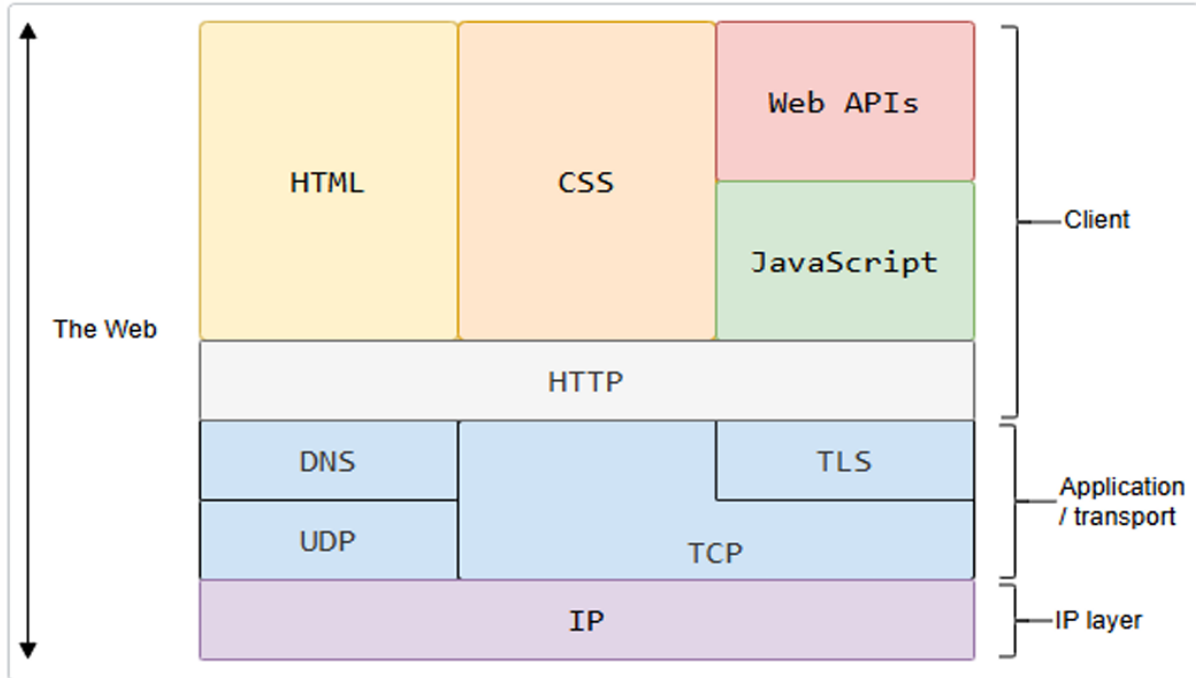
و يقوم بـ:

تشفير البيانات حتى لا يتمكن احد من قراءتها اثناء النقل و التحقق من هوية الموقع (أي ان السيرفر حقيقي و ليس مزيفاً )

(مع امان) HTTP + TLS = HTTPS

## ملاحظة مهمة:

من الممكن نظريا ان يستخدم HTTP أي بروتوكول نقل موثوق اخر غير TCP , لكن عمليا TCP هو المستخدم دائما لأنه الأكثر ثباتاً و موثوقية.



## مكونات نظام HTTP و خصائصه الأساسية:

## 1. العميل (User-Agent / Client):

هو البرنامج الذي يتصرف نيابة عن المستخدم و يقوم بإرسال طلبات HTTP الى السيرفر غالبا يكون:

- المتصفح (Browser): مثل Chrome, Firefox, Edge.
- برامج للمطورين: مثل Postman أو روبوتات محركات البحث تستخدم للتجريب قبل الاستخدام الحقيقي.

## 2. الخادم (Server):

هو الجهة التي تستقبل الطلبات و تعالجها ثم ترسل الرد المناسب، قد يكون هذا الرد صفحة HTML أو ملف صورة أو نتيجة بحث.

## 3. الوسطاء (Proxies):

هي أجهزة أو برامج تتوسط بين العميل و السيرفر و تقوم بعدة وظائف.



## ⚡ اهم وظائف ال proxies:

| الوظيفة        | الشرح                                                                                                                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Caching        | تقنية لتخزين البيانات مؤقتاً في مكان تخزين عالي السرعة (ذاكرة التخزين المؤقت) لتسريع الوصول إليها في المرات القادمة و تقليل الضغط على السيرفر.                                                                                                   |
| Filtering      | فحص او حجب محتوى معين مثل رقابة الاهل او فحص الفيروسات...                                                                                                                                                                                        |
| Load Balancing | عملية توزيع حركة مرور الشبكة الواردة (مثل طلبات المستخدمين) بالتساوي على مجموعة من الخوادم أو الموارد لدعم تطبيق واحد. يهدف ذلك إلى منع خادم واحد من التحميل الزائد، مما يحسن أداء التطبيق ويضمن توفره واستجابته السريعة، ويقلل من زمن الاستجابة |
| Authentication | التأكد من هوية المستخدم قبل منحه الوصول.                                                                                                                                                                                                         |
| Logging        | حفظ سجل بالطلبات لأغراض الأمان و المراقبة.                                                                                                                                                                                                       |

## الخصائص الأساسية ل HTTP (Basic Aspects of HTTP):

## 1. HTTP is stateless:

- كل طلب يُرسل بشكل مستقل تماماً عن الطلب الذي قبله.
- السيرفر لا يتذكر المستخدم بعد ان يرد على الطلب.

**مثال:** لو فتحت صفحة و سجلت دخول ، ثم فتحت صفحة ثانية ، السيرفر لا يعرف انك نفس الشخص . الا اذا استخدمنا أدوات إضافية مثل Cookies.

## 2. HTTP Cookies تعالج مشكلة ال Stateless:

**الـ Cookies:** هي ملفات صغيرة يرسلها السيرفر الى المتصفح ، ليحتفظ فيها بمعلومات المستخدم (مثل اسمه او رقم الجلسة ..).

- بهذه الطريقة يمكن للموقع ان يتذكرك عند كل طلب جديد.
- مثال:** عند تسجيل دخول على موقع ويب، و البقاء داخل الحساب حتى بعد إغلاق الصفحة. سنتعمق فيما بعد بال Cookies ..

## 3. التحكم في الاتصال (Connection Control):

- الاتصال يُدار في طبقة النقل (Transport Layer) و ليس ضمن HTTP نفسه.
- HTTP لا يحتاج ان يكون "connection-based" لكنه يحتاج ان يكون موثوق (أي لا يفقد البيانات).
- لذلك يعتمد على TCP لانه يضمن وصول البيانات كاملة، بخلاف UDP الذي لا يضمنها.

**الخلاصة:** HTTP نفسه لا يتعامل مع التفاصيل الفيزيائية للاتصال، بل يعتمد على TCP ليهتم بإرسال و استقبال البيانات بشكل آمن و موثوق.

بعد أن فهمنا مكونات النظام (العميل – السيرفر – الوسطاء)، الآن سوف نرى كيف يكون التواصل فعلياً بينهم خطوة بخطوة.

### هنا يكون مفهوم HTTP Flow، أي تدفق العمل داخل البروتوكول.

#### • الخطوة 1: Open a TCP connection

أولاً العميل (client) يفتح اتصال TCP مع السيرفر، هذا الاتصال هو الذي ينقل طلب HTTP و الرد عليه. يمكن للعميل (Client) أن يفتح اتصالاً جديداً، أو يعيد استخدام اتصال موجود مسبقاً، أو يفتح عدة اتصالات TCP مع الخوادم (Servers).

#### • الخطوة 2: send a HTTP message

المتصفح يرسل رسالة الطلب (request) للسيرفر. قبل إصدار HTTP/2 كانت الرسائل مقروءة نصياً (human-readable). في HTTP/2 أصبحت الرسائل مشفرة ضمن frames (إطارات) لجعل النقل أسرع و أكثر اماناً. رغم هذا التغيير، مبدأ العمل نفسه بقي ثابتاً: طلب يُرسل - > رد يُستقبل.

#### • الخطوة 3: Read the response

السيرفر يستقبل الطلب، يعالجه، ثم يرسل استجابة (response) تحتوي على كود الحالة (status code) و البيانات او الصفحة المطلوبة.

**مثال:** إذا أرسلت طلباً لصفحة موقع سيرجع السيرفر برد يحتوي على كود الحالة "200 OK" و body فيه محتوى الصفحة.

#### • الخطوة 4: Close or Reuse

❖ بعد استلام الرد، المتصفح ممكن:

- يغلق الاتصال (كان هذا في HTTP/1.0)
- او يعيد استخدامه لطلبات أخرى (في HTTP/1.1 و HTTP/2)

❖ إعادة الاستخدام توفر الوقت وتقلل التأخير (latency)

### مثال عملي على HTTP flow:

لنفرض ان المستخدم كتب في المتصفح العنوان التالي `https://developer.mozilla.org` في هذه الحالة، المتصفح سيُرسل طلب HTTP (request) إلى السيرفر كالتالي:

HTTP

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

## هذا الطلب يعني ان المستخدم يريد الصفحة الرئيسية للموقع و باللغة الفرنسية اذا كانت متاحة.

بعدها يقوم السيرفر بمعالجة الطلب و يرسل استجابة (response) تحتوي على المعلومات المطلوبة:

HTTP

```
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2010 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html

<!doctype html>... (here come the 29769 bytes of the requested web page)
```

### • كل رسالة تتألف من 4 أجزاء رئيسية:

|             |                                                               |
|-------------|---------------------------------------------------------------|
| Start -Line | اول سطر في الرسالة (يحدد نوع الطلب او الكود)                  |
| Headers     | معلومات إضافية عن الرسالة مثل نوع البيانات او الترميز         |
| Empty Line  | سطر فارغ يفصل بين الرأس والمحتوى                              |
| Body        | جسم الرسالة الذي يحتوي على البيانات نفسها (مثلا محتوى الصفحة) |

## ↪ أنواع الرسائل (Request / Response):

### 1. رسالة الطلب (HTTP request):

❖ تتكون رسالة الطلب عادة من:

■ طريقة الطلب (method):

تحدد نوع العملية التي يريد العميل تنفيذها مثل GET , POST , PUT , DELETE ، يوجد عمليات أخرى ولكن هذه أشهرها والأكثر استخداماً

■ المسار (path):

هو عنوان المورد المطلوب (مثل index.html/).

■ اصدار البروتوكول (protocol version):

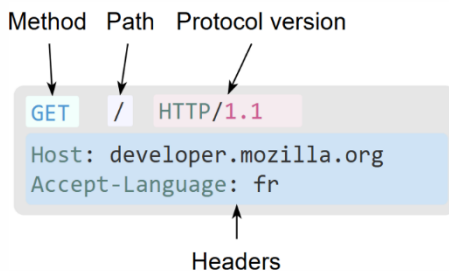
مثل HTTP/1.1 او HTTP/2.

■ Headers:

تحتوي على معلومات إضافية مثل نوع البيانات المطلوبة او اللغة.

■ Body (اختياري):

يستخدم لإرسال بيانات إضافية (مثل بيانات نموذج تسجيل الدخول).



## 2. رسالة الاستجابة (HTTP Response):

❖ تتكون رسالة الاستجابة من :

■ :Start line

■ يحتوي على إصدار البروتوكول و **Status code** و **status message**.

■ :Headers

■ معلومات إضافية عن الرد (مثل نوع المحتوى او تاريخ اخر تعديل).

■ :Empty Line

■ يفصل بين الرؤوس و المحتوى.

■ :Body

■ يحتوي على المورد المطلوب (صفحة HTML مثلاً).

Protocol version    Status code    Status message

```

HTTP/1.1    200    OK
date: Tue, 18 Jun 2024 10:03:55 GMT
cache-control: public, max-age=3600
content-type: text/html

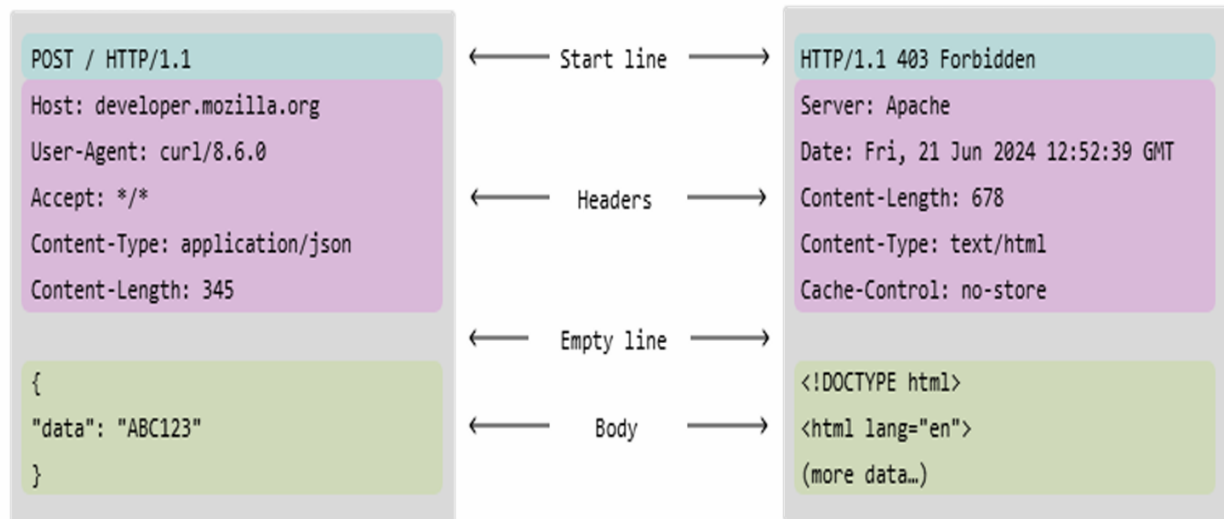
```

↑  
Headers

## The structure of HTTP Messages

Request

Response



- كل رسائل HTTP ، سواء كانت Request أو Response تتبع نفس البنية.
- ال **start line** تحدد نوع العملية او نتيجة الطلب.
- ال **headers** تحمل معلومات إضافية **metadata**.
- **Empty line** يفصل بين الراس و المحتوى.
- **Body** يحتوي على البيانات (مثل صفحة HTML او بيانات POST).

## :some HTTP methods

- **GET (method request)**: تطلب نسخة من المورد المحدد
- **HEAD (method asks)**: يستخدم هذا الطلب للحصول على نفس الاستجابة كما في GET و لكن بدون Body
- أي انه يطلب فقط Headers لمعرفة معلومات عن المورد مثل الطول او النوع دون تحميل المحتوى نفسه.
- **POST (method submits)**: يستخدم هذا الطلب لإرسال بيانات الى الخادم لمعالجتها (مثل نموذج تسجيل) في اغلب الأحيان يؤدي الى تغيير في حالة السيرفر او البيانات الموجودة عليه.
- **مثال:** عندما تضغط "تسجيل دخول" يُستخدم POST لإرسال البيانات.
- **PUT (method replaces)**: يستخدم هذا الطلب لاستبدال المورد الحالي بالكامل بالمحتوى الجديد المُرسَل من العميل بمعنى اخر: يتم تحديث او إعادة كتابة البيانات الحالية بمحتوى جديد تماما.
- **DELETE (method deletes)**: يستخدم هذا الطلب لحذف مورد معين.

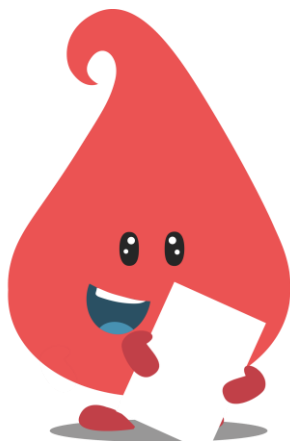
## :Some HTTP Headers

- **Accept**: في الطلب و الاستجابة يشير الى أنواع المحتوى التي يستطيع المرسل فهمها .
- **Accept-Encoding**: في الطلب و الاستجابة يشير الى ترميز المحتوى (عادة خوارزمية ضغط) التي يستطيع المرسل فهمها.
- **Age**: في الاستجابة يشير الى الزمن بالثواني الذي بقي فيه الكائن داخل ذاكرة التخزين المؤقت (cache).
- **ETag (entity tag)**: في الاستجابة هو معرف لإصدار محدد من المورد.

## :HTTP Response Status Codes

❖ تشير أكواد حالة استجابة HTTP إلى ما إذا كان طلب HTTP محدد قد تم تنفيذه بنجاح ام لا:

- Informational responses: 100 – 199
- Successful responses: 200 – 299
- Redirection messages: 300 – 399
- Client error responses: 400 – 499
- Server error responses: 500 – 599



## Examples of Response status codes

- 200 OK: نجاح الطلب
- 201 Created: نجاح الطلب، و تم انشاء مورد جديد نتيجة لذلك ، يُستخدم هذا عادة بعد طلبات POST أو بعض طلبات PUT.
- 204 No Content: لا يوجد محتوى لإرساله لهذا الطلب ، و لكن ال Headers مفيدة.
- 301 Moved Permanently: تم نقل المورد بشكل دائم.
- 304 Not Modified: يُستخدم لأغراض التخزين المؤقت (caching) ، يُخبر العميل ان الاستجابة لم يتم تعديلها.
- Bad Request: طلب غير صالح.
- 401 Unauthorized: يجب على العميل ان يُجري عملية مصادقة (تسجيل دخول ) للحصول على المصادقة.
- 403 Forbidden: لا يملك العميل صلاحيات الوصول الى المحتوى المطلوب.
- 404 Not Found: لا يستطيع الخادم العثور على المورد المطلوب.
- 502 Bad Gateway: يشير هذا الخطأ الى ان الخادم أثناء عمله كبوابة للحصول على استجابة من خادم اخر تلقى استجابة غير صالحة.

## تطور بروتوكول HTTP:

## 1. HTTP/1.0 - اول اصدار:

- اول نسخة من البروتوكول (ظهرت في بداية التسعينات)
- كان يرسل مع كل طلب معلومة عن الإصدار.
- اضافوا سطر خاص في بداية الرد يسمى Status Line.
- قدّم فكرة Headers في الطلب.
- سمح بإرسال ملفات من أنواع مختلفة، ليس فقط صفحات HTML.

## 2. HTTP/1.1 - الإصدار القياسي:

- تم تحسين الأداء: يمكن إعادة استخدام نفس الاتصال لأكثر من طلب.
- أضيفت ميزة Pipelining: ارسال طلب جديد قبل انتهاء الرد السابق.
- أضيفت أدوات جديدة للتحكم في ال cache.
- أصبح العميل والسيرفر قادرين على الاتفاق على نوع البيانات.
- تم تقديم رأس Host، مما سمح بتعدد المواقع على نفس ال IP.

## 3. SSL و TLS - الأمان:

- سنة 1994 شركة Netscape اضافت طبقة امان اسمها SSL.
- هذه الطبقة تقوم بتشفير البيانات بين المتصفح والسيرفر.
- لاحقا تم تحسينها وأصبحت المعيار العالمي باسم TLS.
- بفضلها ظهر HTTPS.

## 4. نموذج REST (عام 2000):

- أسلوب جديد لاستخدام HTTP اسمه REST (Representational State Transfer).
- يعتمد على الوصول الى روابط (URIs) باستخدام الطرق العادية مثل GET و POST.
- مكن المواقع من انشاء واجهات API لتبادل البيانات.
- العيب: كل موقع صمم واجهته بطريقته (ما كان في معيار موحد).

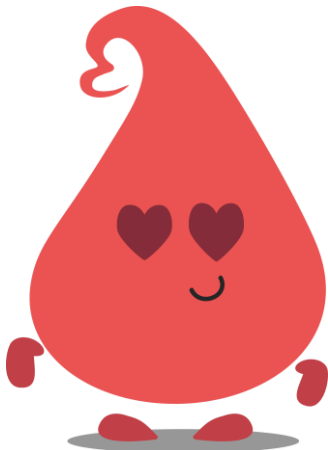
## 5. HTTP/2 - أداء و سرعة اعلى:

- تم تطويره ليعالج ببطء HTTP/1.1.
- أصبح البروتوكول ثنائي (Binary) بدل النصي.
- صار يدعم ارسال عدة طلبات بنفس الوقت على اتصال واحد (Multiplexed).
- صار يضغط رؤوس الرسائل (Header Compression) لتسريع التحميل.

## 6. HTTP/3 - الإصدار الحديث:

- نفس أفكار HTTP السابقة لكن يستخدم بروتوكول جديد اسمه QUIC بدل TCP.
- QUIC يعمل فوق UDP و يعالج كل تدفق (stream) بشكل مستقل.
- هذا يعني: حتى لو تعطل جزء من الاتصال، الباقي يكمل بشكل طبيعي.
- بحلول عام 2022 أكثر من ربع الانترنت اصبح يستخدم HTTP/3.

ملاحظة: محاضرات الأربيسيز من الممكن أن تكون موسعة بالمعلومات.. لأهميتها وتبسيطها...



انتهت المحاضرة

<The end>